Enrique Georg Zbinden

Informatikpraktikum bei Tatsson.com

Konzeption und Entwicklung des WordPress-Plugins PropVista

29. Juni bis 18. September 2026

12 Wochen, 100 %, Remote

Mein Einstieg bei Tatsson

Plan: Projektumfang mit Praktikumsgeber abgestimmt

Tatsson.com, Tbilisi

Webdesign und Webentwicklung
Betreuung: Tato-Shalva Makhashvili
12 Wochen, 100 %, remote

Mein Ausgangspunkt

Python, C und Java aus dem Studium

Neu: PHP und die Entwicklung innerhalb von WordPress

Kundenanforderungen in eine begrenzte technische Lösung übersetzen

Erkenntnis: Ich musste Programmierwissen in einen konkreten Projektkontext übersetzen.

Unternehmenskontext: Projektbeschreibung. Vorkenntnisse: eigene Angaben.

Kundenbedarf und Projektauftrag

Plan: Projektumfang mit Praktikumsgeber abgestimmt

Wiederkehrendes Problem

Gebäudepläne sollen Einheiten mit Informationen verknüpfen.

Einzellösungen erschweren Wiederverwendung und Wartung.

Meine Kernanforderungen

Plan- und Einheitendaten pflegen
Plan auf einer Seite anzeigen
Einheit über einen Marker auswählen
Zugehörige Informationen anzeigen

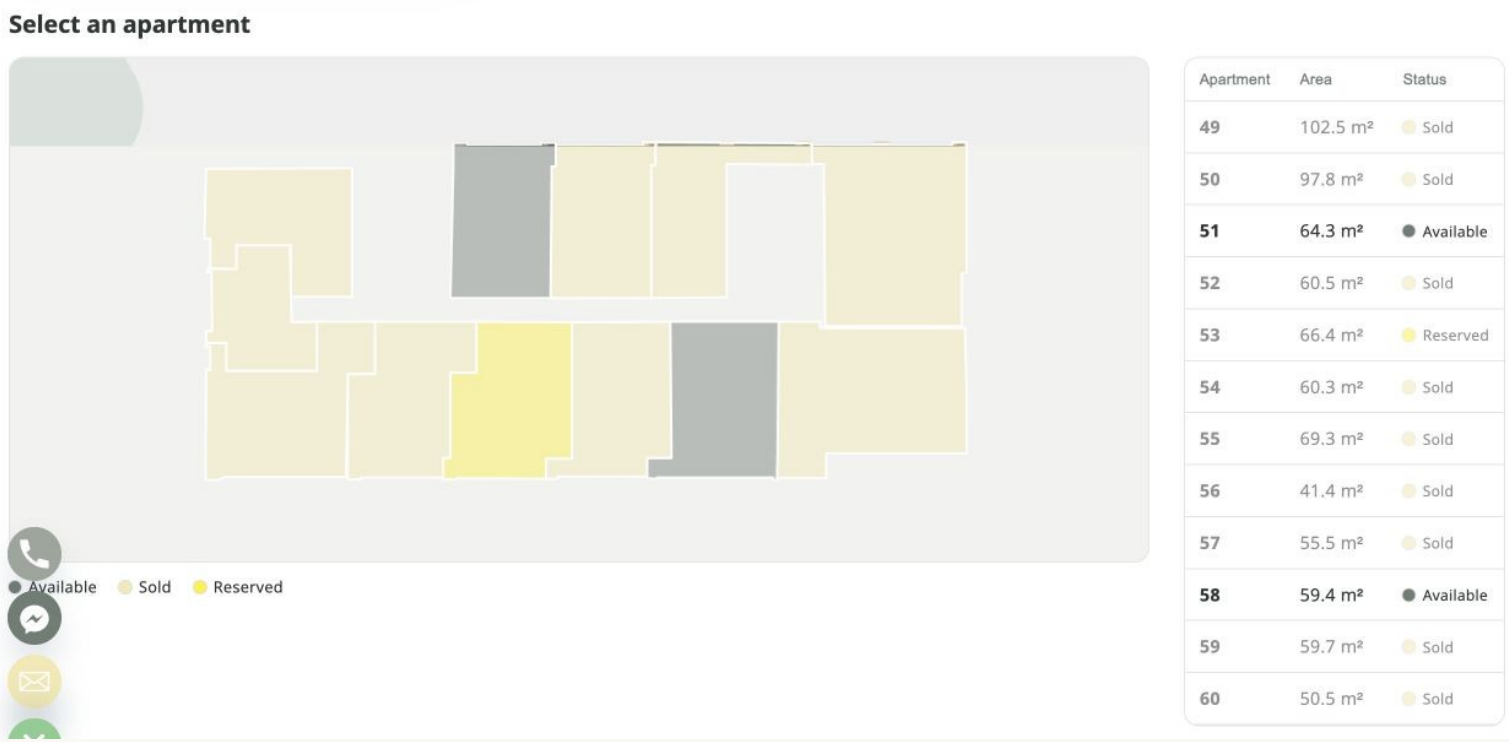
Ergebnisziel: interner Plugin-Prototyp mit Tests, Dokumentation und Übergabe

Erkenntnis: Ein verständliches Nutzerziel hilft, den Umfang einer Lösung zu begrenzen.

Projektumfang und erste Recherche rückblickend im Zwischenbericht beschrieben.

Praxisbeispiel aus der Recherche

Plan: Grundverständnis WordPress erarbeitet



Untersucht

Auswahl einer Einheit

Zuordnung der Daten

Übersichtlichkeit

Erkenntnis: Eine Referenz hilft, die Benutzerführung konkret zu diskutieren.

Praxisreferenz Impel Verde. Aufnahme 30.09.2026. Recherche lief teilweise parallel.

Wirtschaftliche Einordnung der Plugin-Idee

Plan: Grundverständnis WordPress erarbeitet

Ansatz	Möglicher Vorteil	Aufwand und Grenze
Individuell je Kunde	Genaue Anpassung an den Kunden	Entwicklung und Wartung fallen je Projekt an
Drittanbieter-Plugin	Weniger eigene Erstentwicklung	Integration, Abhängigkeit, mögliche Lizenzen
Eigenes Plugin	Gemeinsame Basis für mehrere Projekte	Erstentwicklung, Anpassung und dauerhafte Wartung

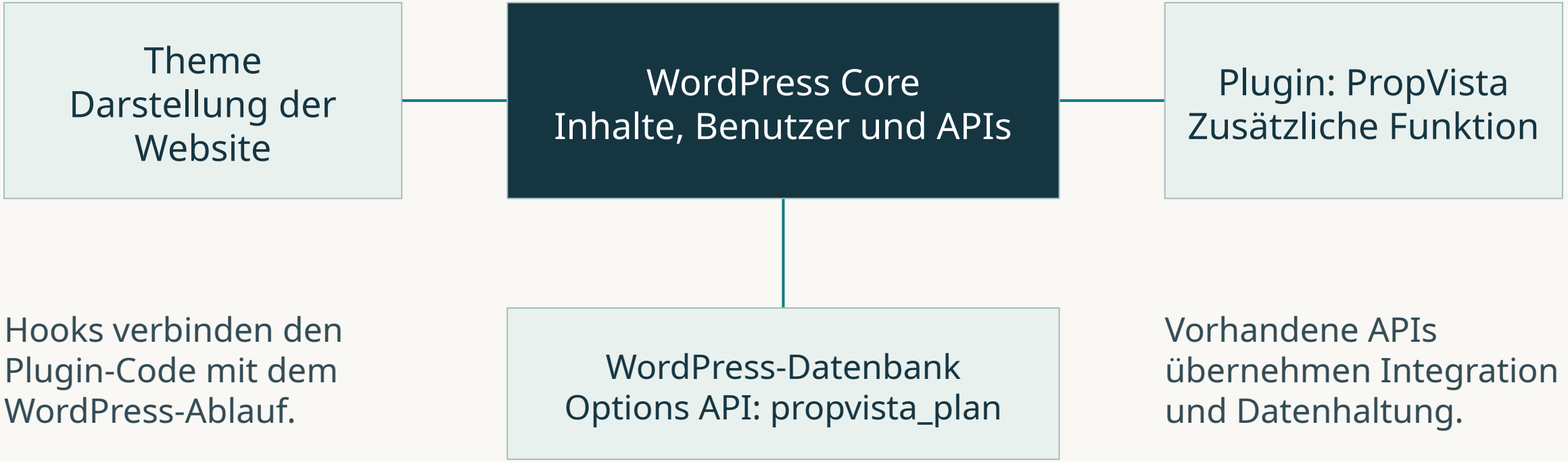
Gesamtkosten = Einführung / Entwicklung + Anpassung
+ Betrieb, Lizenzen und Wartung

Heute eingeordnet: Wiederverwendung allein belegt noch keinen Kostenvorteil.

Nachträgliche qualitative Einordnung der Recherche. Keine damalige Kosten- oder Marktstudie.

WordPress: Core, Theme und Plugin

Plan: Grundverständnis WordPress erarbeitet



Hooks verbinden den Plugin-Code mit dem WordPress-Ablauf.

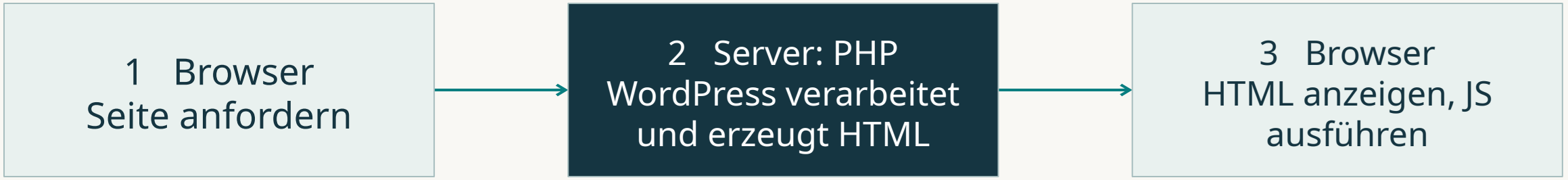
Vorhandene APIs übernehmen Integration und Datenhaltung.

Erkenntnis: Zur Programmiersprache kommt die Ablauflogik des bestehenden Systems hinzu.

Einarbeitung laut Zwischenbericht. Diagramm zeigt den fertigen Prototyp als Rückblick.

PHP und das Client-Server-Modell

Plan: Technische Basis erarbeitet



PHP liest auf dem Server

```
$data = get_option('propvista_plan', false);
```

JavaScript verarbeitet danach die Marker-Auswahl im Browser.
Der gespeicherte Plan bleibt gleich.

Erkenntnis: Ich musste verstehen, wo der Code läuft und welche Ausgabe weitergegeben wird.

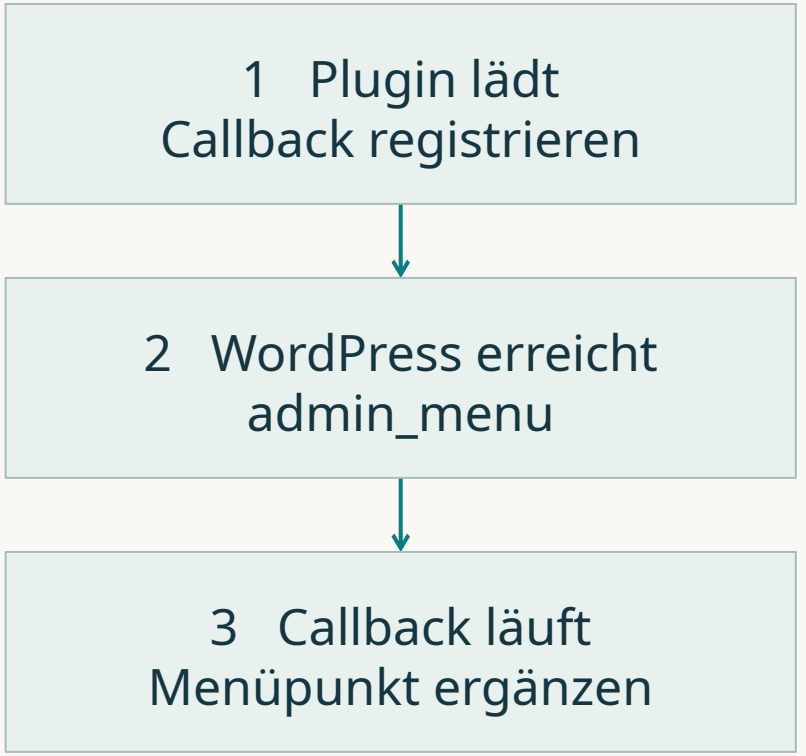
Originalcode aus dem fertigen Prototyp. Frühe Einarbeitung rückblickend berichtet.

Hooks: Registrierung und Ausführung

Plan: Technische Basis erarbeitet

Originalregistrierung

```
add_action(
  'admin_menu',
  'propvista_register_admin_page'
);
```



Die Registrierung hinterlegt den Callback.
WordPress bestimmt seinen Aufrufzeitpunkt.

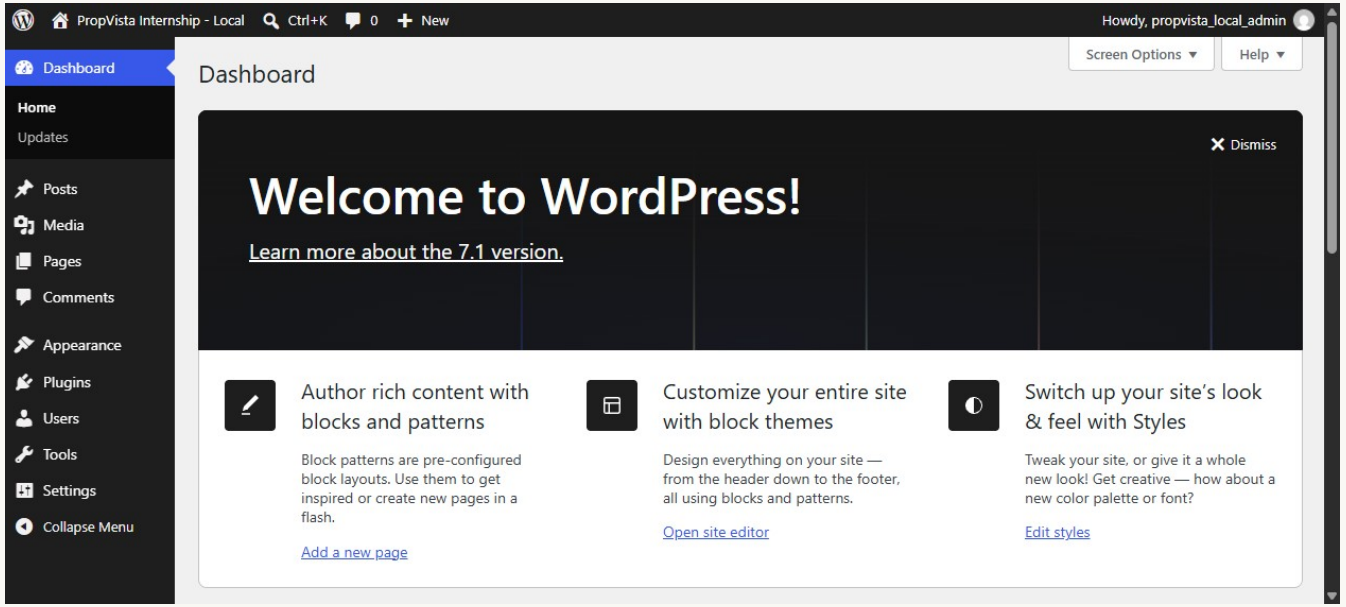
Erkenntnis: Eine Funktion verstehen heisst auch, ihren Aufrufzeitpunkt zu kennen.

Originalregistrierung aus includes/plugin.php. Fertiger Code als Lernbeispiel im Rückblick.

Interne Testumgebung und Arbeitsablauf

Plan: Entwicklungsumgebung lauffähig

Originalaufnahme: WordPress-Dashboard



Praktischer Ablauf

Kleine PHP-Versuche

Plugin aktivieren

Ausgabe und Verhalten prüfen

Erkenntnis: Kleine Versuche halfen mir, das System schrittweise zu verstehen.

Im Zwischenbericht als eingerichtet beschrieben. Originalaufnahme der Nachprüfung am 12.09.

Mockup und Benutzerführung

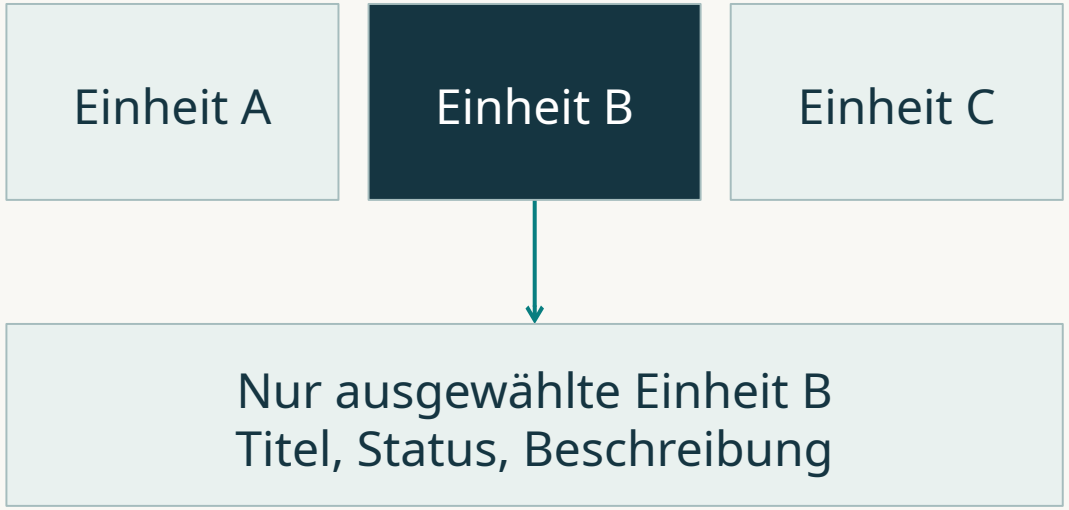
Plan: Konzept mit Praktikumsgeber abgestimmt

Zuerst: alle Informationen



Der Plan konkurriert mit drei Informationsblöcken.

Vereinfacht: Auswahl steuert Details



Klare Verknüpfung von Marker und Einheitsinformation

Erkenntnis: Mehr Informationen verbessern die Bedienung nicht automatisch.

Schematische Rekonstruktion der berichteten Entscheidung. Original-Mockup nicht verfügbar.

Kernfunktion und Priorisierung mit Tato

Plan: Konzept mit Praktikumsgeber abgestimmt

Genehmigter Auftrag

Einfache Struktur für Plan und Einheiten
Anklickbare Marker
Minimale Administration

Späterer konkreter Prototyp

Ein gespeicherter Plan
Drei vorbereitete Einheiten und Marker
Bearbeitung bestehender Einträge

Mit Tato priorisiert

Die nächste prüfbare Kernfunktion
vor zusätzlichen Komfortfunktionen

Die Zahl drei ist keine Vorgabe.
Weitere Pläne und ein visueller Editor
bleiben spätere Erweiterungen.

Erkenntnis: Ich musste Erweiterungsideen an ihrem Beitrag zum vereinbarten Nutzerziel prüfen.

Priorisierung im Zwischenbericht beschrieben. Gezeigt: spätere konkrete Prototypausgestaltung.

Plugin-Grundgerüst und Zuständigkeiten

Plan: Plugin wird von WordPress erkannt

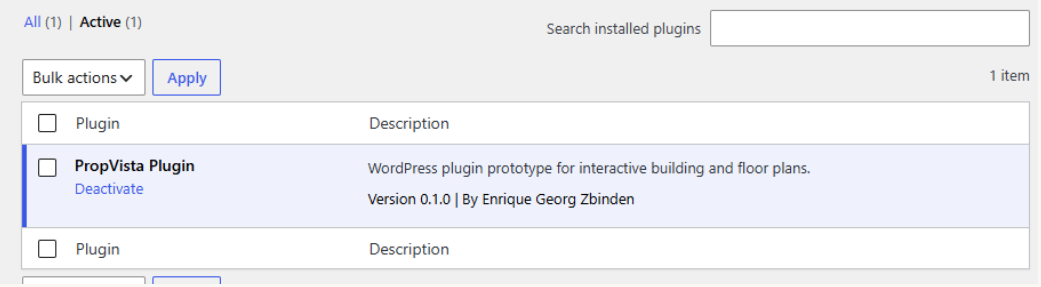
Einstieg und weitere Aufgaben

propvista-plugin.php
Metadaten und Plugin-Einstieg

includes
Verwaltung, Speicherung, Shortcode

assets
CSS und JavaScript im Browser

WordPress erkennt das Plugin



Aktivierung und Ausgabe prüfen
Die Aufnahme zeigt den späteren Stand.

Erkenntnis: Die Dateistruktur hilft, Aufgaben und Fehlerstellen getrennt zu untersuchen.

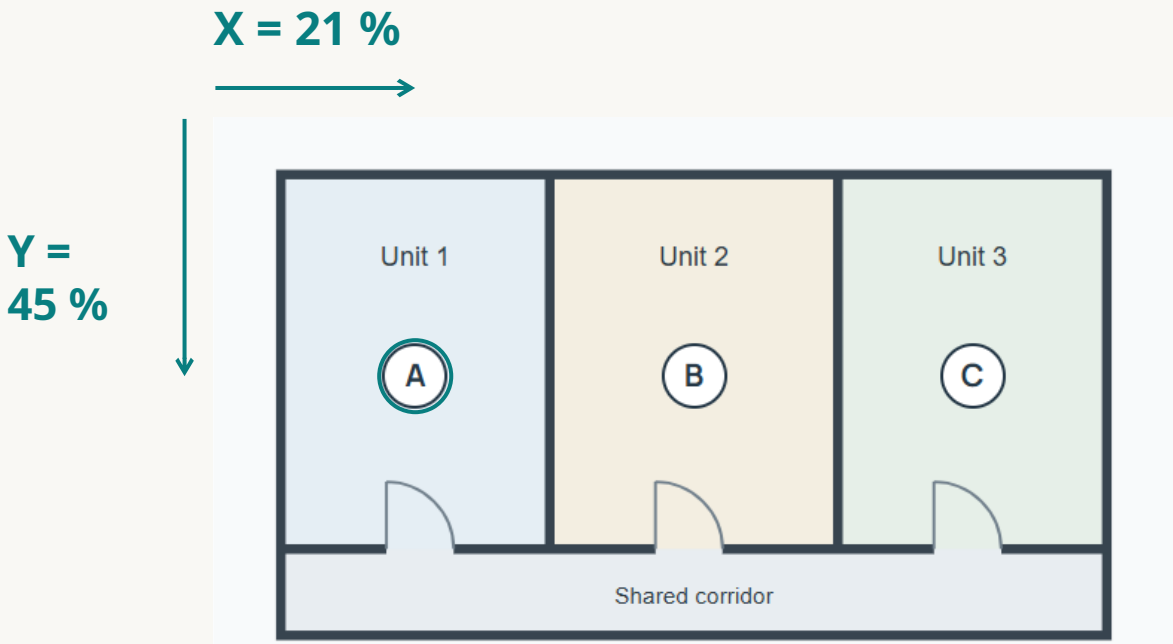
Grundgerüst laut Zwischenbericht. Originalaufnahme des aktivierten Plugins am 13.09.

Datenstruktur und relative Markerpositionen

Plan: Grunddaten können erfasst oder vorbereitet werden

Gespeicherte Struktur

Plan mit Bildreferenz
Einheiten mit stabiler ID
Marker: unit_id, x, y, label



Marker A: unit_id = unit-1
X = 21 % der Bildbreite
Y = 45 % der Bildhöhe

Erkenntnis: Eine kleine Darstellungsfrage beeinflusst Datenmodell und Administration.

Entscheidung im Zwischenbericht. Originalaufnahme 13.09.2026 mit ergänzten Masspfeilen.

Kontrollpunkt: Zwischenbericht

Plan: Grunddaten können erfasst oder vorbereitet werden

Als erarbeitet beschrieben

Einarbeitung und Konzept
Plugin-Grundgerüst
Plan-, Einheits- und Markerstruktur

Als nächste Schritte vorgesehen

Administration
Frontend und Interaktion
Tests und Dokumentation

Der vollständige Ablauf war noch offen.

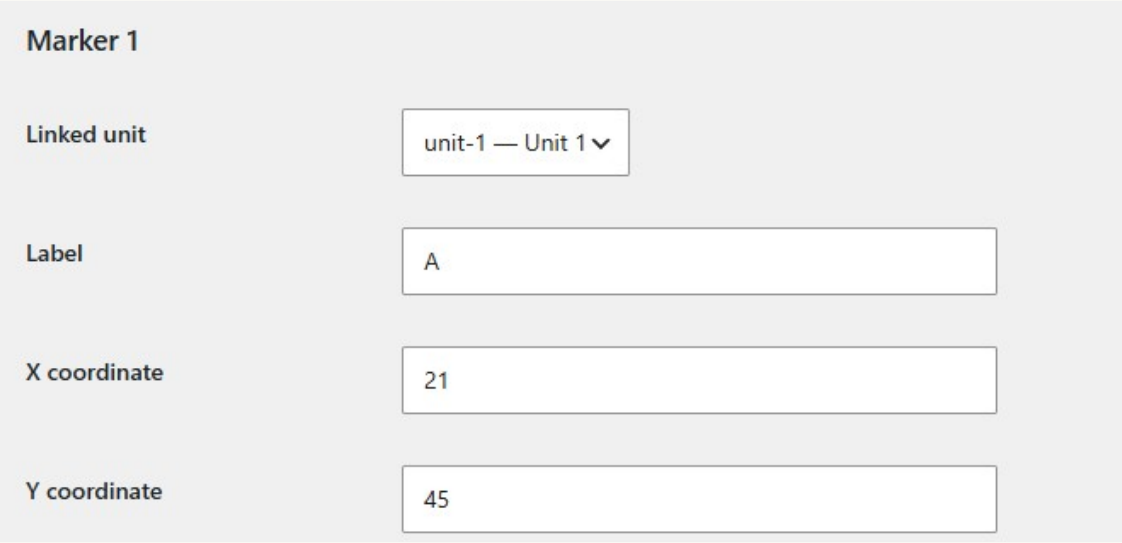
Erkenntnis: Der Zwischenstand braucht eine klare Grenze zwischen vorbereitet und funktionsfähig.

Berichtsstand Ende Woche 7. Der Schlussbericht nennt dafür das Dokumentdatum 20.08.2026.

Administration und geprüftes Speichern

Plan: Basisinformationen können verwaltet werden

Marker bearbeiten



Einheit unit-1, Label A, Position 21 / 45 %

Schreiben und Rücklesen

Berechtigung und Nonce prüfen
Vollständige Eingaben normalisieren
Gespeicherten Zustand vergleichen

```
update_option('propvista_plan', $data);
$stored = get_option('propvista_plan', false);
```

Vereinfachter Prinzipcode. Im Original: Normalisierung und Vergleich im Save-Handler.

Erkenntnis: Speichern bedeutet für mich auch, das Ergebnis kontrollieren zu können.

IST: umgesetzt und geprüft am 12.09. Originalaufnahme 13.09. Code rechts vereinfacht.

Speicherweg und späterer Seitenaufruf

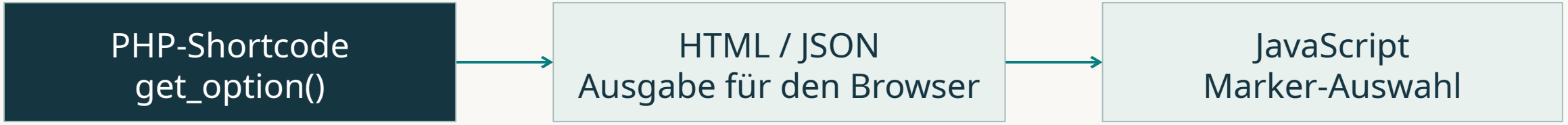
Plan: Basisinformationen können verwaltet werden

A Administration und Speicherung



Gemeinsamer Datensatz: WordPress-Option propvista_plan

B Späterer Seitenaufruf und Interaktion



Erkenntnis: Schreiben, Lesen und Browserinteraktion erfüllen unterschiedliche Aufgaben.

Datenfluss des fertigen Prototyps. Administration am 12.09., Frontend und Interaktion am 13.09.

Frontend-Integration mit einem Shortcode

Plan: Plan wird im Frontend angezeigt

WordPress-Seite mit Shortcode



Originalregistrierung

```
add_shortcode(
  'propvista_plan',
  'propvista_render_plan'
);
```

Der Callback liefert HTML.
Die Seite nutzt denselben Plan.
Der Shortcode hat keine Parameter.

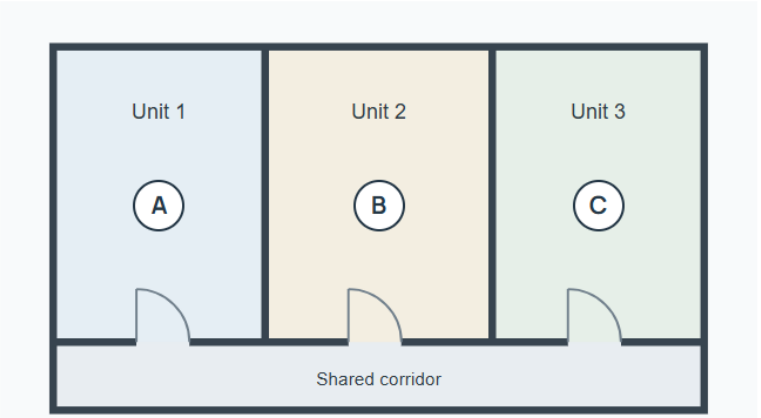
Erkenntnis: Ich konnte die Funktion in die vorhandene Seite integrieren.

IST: Frontend am 13.09. umgesetzt und geprüft. Originalaufnahme des Seiteneditors.

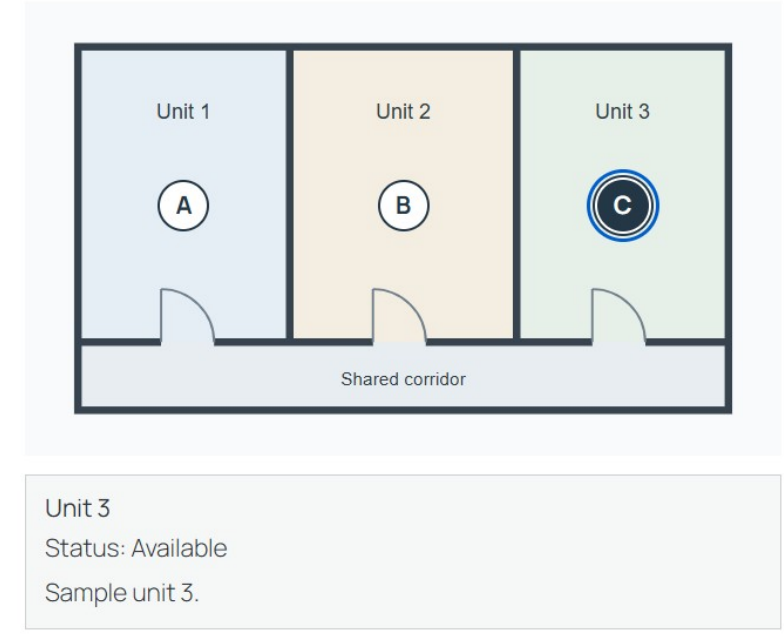
Kernfunktion: Auswahl einer Einheit

Plan: Kernfunktion im Prototyp nutzbar

1 Plan ohne Auswahl



2 Klick auf C: Informationen



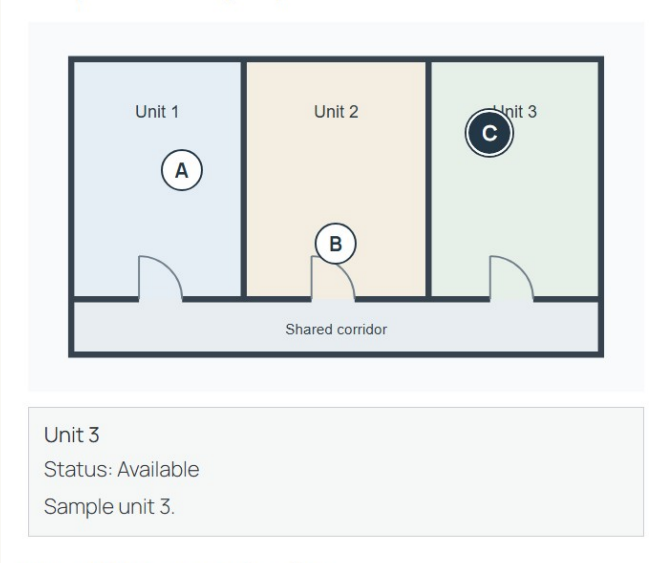
Erkenntnis: Das Nutzerziel wurde an einem kleinen vollständigen Ablauf prüfbar.

IST: Interaktion am 13.09. geprüft. Zwei Originalaufnahmen desselben konfigurierten Plans.

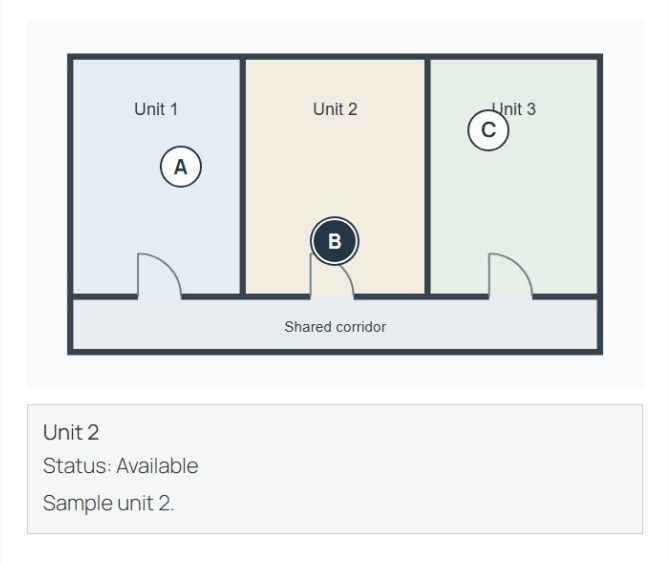
Unabhängiger Zustand je Einbindung

Plan: Kernfunktion im Prototyp nutzbar

Einbindung 1: C / Unit 3



Einbindung 2: B / Unit 2



let selectedButton = null;

Lokal in initializePlan(plan)

Erkenntnis: Wiederverwendbarer Code braucht klar begrenzte Zuständigkeiten für den Zustand.

Original-Nachtest 13.09.2026. Der Bericht beschreibt die anfängliche Kopplung.

Tests, Ergebnisse und bekannte Grenzen

Plan: Prototyp stabilisiert

Prüffrage	Verfahren	Beobachtung
Bleiben Daten erhalten?	Speichern, wiederladen und separat rücklesen	Persistenz bestätigt
Passt Marker zu Einheit?	A, B und C auswählen	Zugeordnete Daten angezeigt
Bleiben Instanzen getrennt?	Unterschiedliche Auswahl auf derselben Seite	Informationsbereiche unabhängig
Was passiert bei Randfällen?	Ungültige Daten, Tastatur und Kollision	Kontrollfälle geprüft, Kollision bleibt bekannte Grenze

Deckungsgleiche Marker können sich bei der Mausbedienung verdecken.

Erkenntnis: Ich musste Erfolgskriterien und verbleibende Grenzen ausdrücklich unterscheiden.

IST: integrierte Prüfung am 13.09. Keine breite Browser-/Theme-Matrix oder vollständige WCAG-Prüfung.

Fehleranalyse des Testablaufs

Plan: Prototyp stabilisiert

Fehlversuch und Diagnose

Die Speichern-Schaltfläche lag ausserhalb des sichtbaren Bereichs. Der Testtreiber löste sie nicht korrekt aus.

Scrollen, erneut klicken, wiederladen und Daten rücklesen.

Originalauszug: Prüfprotokoll


```
{
  "id": "native-save-after-explicit-scroll",
  "time": "2026-09-13T09:14:33.249Z",
  "expected": true,
  "actual": true,
  "result": "PASS"
}
```


Plugin-Code in dieser Fortsetzung unverändert.

Erkenntnis: Ein fehlgeschlagener Test verlangt zuerst eine Diagnose der Ursache.

Originalauszug aus browser-workflow-resume.json vom 13.09.2026.

Dokumentation, Übergabe und Soll-Ist

Plan: Prototyp abgeschlossen und übergeben

README: Installieren und verwenden

Publish and use the plan

1. Create a WordPress page, […]

[propvista_plan]

3. Publish the page and open its
   frontend view. […]

Originalauszug aus README.md, gekürzt

Tatsächlicher Verlauf

12./13.09.	Umsetzung und Prüfungen
13.09.	Paket und Anleitung geprüft
17.09.	Übergabe laut Schlussbericht
18.09.	Nachbesprechung

Integration später als geplant.
Übergabe in der Abschlusswoche.

Erkenntnis: Der Abschluss muss einen klaren Umfang, Prüfstand und offene Grenzen vermitteln.

Vorbereitung: README und Paketprüfung 13.09. Übergabe 17.09. und Debriefing 18.09. laut Schlussbericht.

Fazit: Technik, Arbeitsweise und Ausblick

Plan: Prototyp abgeschlossen und übergeben

Fachlich gelernt	Arbeitsweise	Ergebnisse selbst prüfen
PHP, WordPress und Browserinteraktion als gemeinsamen Ablauf verstehen	Früher einen vollständigen Kernablauf entwickeln Erweiterungen priorisieren Feedback von Tato nutzen	KI-Zusammenfassung vermischte geplante und umgesetzte Funktionen. Abgleich mit Code und Paket

Ausblick

Flexiblere Datenpflege, breitere Tests und Rückmeldungen echter Anwender

Vielen Dank